Project Context — Deepak Keshav Warude's Portfolio

This document exists so that any AI assistant (or human) picking up this project later has full context without needing to re-derive it from scratch. Read this before making changes.

What this is

A personal portfolio website for **Deepak Keshav Warude**, a Computer Engineering student (K. K. Wagh Institute of Engineering Education & Research, Nashik), built to help him land software engineering interviews. The site was built iteratively, section by section, with the owner reviewing and correcting each step.

Live links

- **Production site**: https://portfolio-deepak-warude.vercel.app
- **GitHub repo**: https://github.com/dkwarude-cell/Portfolio_Deepak_Warude
- **Vercel project**: `deepakwarude/portfolio-deepak-warude` (connected to the GitHub repo — every push to `main` auto-deploys)
- Owner's GitHub: https://github.com/dkwarude-cell
- Owner's LinkedIn: https://www.linkedin.com/in/deepak-warude-621592394
- Owner's email: deepakwarude009@gmail.com

Tech stack

- **Next.js 16** (App Router, Turbopack, React 19)
- **TypeScript**
- **Tailwind CSS v4** (CSS-based `@theme`, no `tailwind.config.js`)
- **shadcn/ui**, but on the newer **Base UI** primitives (`@base-ui/react`), not Radix — see gotchas below. The `shadcn` CLI package itself lives in `devDependencies` (it's a codegen tool run via `npx shadcn add ...`, never imported by app code) — don't move it back to `dependencies`.
- **Framer Motion v14** for all animation/scroll effects
- **next-themes** for dark/light toggle (dark is the default theme)
- **lucide-react** for icons, with two hand-rolled SVG icons for GitHub/LinkedIn (see gotchas)

This stack was deliberately chosen (over simpler static templates) partly because it mirrors the owner's actual resume skills (React, TypeScript, Tailwind, Next.js-adjacent), making the portfolio itself a credible demonstration of ability, and partly because the owner explicitly asked for "best technologies" and later for an "advanced, futuristic" motion-driven UI.

Design direction (why it looks the way it does)

This was informed by a deep-research pass (see `research_notes/` and `reports/` locally — **gitignored**, not in the repo) comparing engineer portfolios with great UI/UX for job-hunting purposes. Key conclusions that shaped this build:

- Minimal, fast, content-forward sites (e.g. Brittany Chiang's, Lee Robinson's) win hiring attention; flashy Awwwards-style 3D showcases (e.g. Bruno Simon's) win design awards but can read as "not an engineer" to a hiring audience.
- Structure: identity/headline above the fold → 3–6 strong projects with problem/role/stack/result framing → resume/GitHub/LinkedIn links → a clear single contact CTA.
- Dark-mode-first, restrained accent color (not pure grayscale, not garish), avoid pure black/white for contrast comfort.
- A broken "live demo" link is worse than no link at all — every link in `src/lib/data.ts` should be verified to actually resolve before shipping.

Partway through, the owner asked for the UI to feel more "advanced/futuristic" with scroll-reactive motion, which is why the site now also has: a scroll progress bar, cursor-following glow (desktop), scroll-linked parallax in the hero, pointer-reactive 3D tilt on project cards (desktop) / tap-to-reveal glow (touch), and animated section-divider lines. See `src/components/fade-in.tsx`, `tilt-card.tsx`, `cursor-glow.tsx`, `scroll-progress.tsx`.

Content model — how to update the site's content

Almost all editable content lives in one file: `src/lib/data.ts`. It exports:

- `profile` — name, tagline, bio, location, status, email, GitHub/LinkedIn URLs, resume path, avatar path
- `experience` — work history array
- `projects` — array of `Project` objects (`slug`, `name`, `summary`, `period`, `stack`, `bullets`, optional `badges`, `size: "lg"|"md"` for grid span, optional `github` / `live` links)
- `skills` — categorized skill groups
- `education` — degree history
- `achievements` — award/recognition entries, each optionally with an `images: string[]` array (shown as a 2-up photo grid, card auto-spans full width when present)
- `nav` — the header nav links

To add/edit/remove a project, work experience entry, skill, achievement, etc., **edit this file only** — the section components (`src/components/sections/*.tsx`) just render whatever is in `data.ts`.

Images

- Profile photo: `public/images/profile.jpg`
- Achievement photos: `public/images/achievements/*.jpg`
- **Always compress/resize photos before committing.** Source phone photos arrived at 4–16MB each; they were resized to max-width 1600px and JPEG-compressed (quality ~78) with Pillow before committing, landing around 150–450KB each. Don't commit raw phone-camera-sized images.
- When cropping a photo for a specific display aspect ratio (the achievement cards use `aspect-[4/3]` with `object-cover`), check where faces/important content sit in the frame first — `object-cover` center-crops by default and will cut off faces in portrait-oriented photos if you don't pre-crop the source image to roughly match the target aspect ratio. This bit us twice (see commit `8934896`).

Resume PDF

`public/resume.pdf` is a generated PDF (not a design export) — see `scratchpad/build_resume.py` if it still exists locally (it's gitignored, so it may not be present in a fresh clone; regenerate from the resume text if needed). It uses Times-Roman/Times-Bold fonts with justified body text to match the look of the owner's original resume

design, and the contact line (email/LinkedIn/GitHub) has real clickable hyperlinks via ReportLab's `<link href="...">` markup. Keep it to one page — the owner explicitly wants this; tightening `fontSize` / `leading` / `spaceBefore` / `spaceAfter` in that script is how previous overflow was fixed.

Known gotchas / lessons learned (read before touching animation or UI primitives)

- **The shadcn `Button` here uses Base UI, not Radix.** It does **not** support the usual `asChild` prop. Use `render={<a href="...">...</a>}` instead, and pass `nativeButton={false}` when the rendered element isn't a real `<button>`. See `src/components/ui/button.tsx` and any usage in `hero.tsx` / `site-header.tsx`.
- `lucide-react` **(this version) has no brand/logo icons.** `Github`, `Linkedin`, `Twitter`, etc. don't exist in this package version — they were removed upstream. Custom inline SVGs for GitHub/LinkedIn live in `src/components/icons.tsx` (`GithubIcon`, `LinkedinIcon`). Use those, don't try to import brand icons from lucide-react.
- **Framer Motion `initial` / `whileInView` / `animate` targets must stay the same shape across a component's lifetime.** A real, shipped bug (fixed in commit `8d59843`) came from `FadeIn` switching between two differently-shaped animation objects (one with a `filter: blur(...)` key, one without) based on a client-only value (`useCoarsePointer`, via `useSyncExternalStore`) that resolves *after* the first render. Framer Motion captures `initial` once at mount and only animates properties present in the *current* target object — if a property like `filter` is dropped from the target on a later render, it never gets reset and the element stays permanently in its initial (blurred) state. **Rule: if an animation's target shape needs to vary conditionally, always include every property in every variant** (use a no-op value like `blur(0px)` instead of omitting the key).
- `useCoarsePointer()` (`src/hooks/use-coarse-pointer.ts`) detects touch/phone devices via `matchMedia('(pointer: coarse)')`, implemented with `useSyncExternalStore` (not `useState` + `useEffect`, which trips this project's lint rule against synchronous `setState` in an effect body, and is also just the more correct tool for subscribing to external browser state). It's used to:
- Skip the cursor-glow's `pointermove` listener entirely on touch (`cursor-glow.tsx`)
- Swap `TiltCard`'s 3D hover-tilt for a tap-to-reveal glow + press-scale on touch (`tilt-card.tsx`)
- Skip Hero's scroll-linked parallax/infinite float on touch (`hero.tsx`)
- Use cheaper (no-blur) transitions in `FadeIn` on touch
- **Next.js 16's Cache Components feature forbids `new Date()` (and similar non-deterministic calls) during prerendering** — it'll fail the build with "encountered the unstable value". The footer's copyright year is hardcoded (`© 2026 ...`) rather than computed, for this reason.
- **Binary files need `.gitattributes`.** This machine's global git config has `core.autocrlf=true`. Without `.gitattributes` forcing `*.pdf`, `*.png`, `*.jpg`, etc. to `binary`, git's diff/add pipeline can treat them as text and risk CRLF-mangling binary content on commit. The repo's `.gitattributes` handles this — keep it, and add new binary extensions to it if introduced.
- **shadcn/ui's dark theme here uses an accent blue** (`oklch(0.623 0.214 259.815)` dark / `oklch(0.546 0.245 262.881)` light, roughly Tailwind's blue-500/600) layered onto an otherwise grayscale OKLCH palette in `src/app/globals.css`. If re-theming, change `--primary` / `--ring` in both `:root` and `.dark` blocks together.
- **A `mailto:` link alone is not a reliable "contact me" affordance.** If the visitor has no default mail client configured (common on Chrome/Windows), clicking it does nothing visible and looks like a broken icon — this

happened for real and was reported as a bug. Fix: `src/components/email-link.tsx` (`EmailLink`) wraps every email `<a>` so a click also copies the address to the clipboard and shows a brief "Email copied!" tooltip via Framer Motion, in addition to letting the `mailto:` navigation fire normally. All three email touchpoints (`hero.tsx` icon, `contact.tsx` CTA, `contact.tsx` footer link) use this component — don't revert them to plain `<a href="mailto:...">` .

Security

A security review (headers + dependency + secret-scan) was done on this repo; see commit `b22ef0f` . Current state:

- **HTTP security headers** are set in `next.config.ts` via `headers()` : `Content-Security-Policy` , `X-Content-Type-Options` , `X-Frame-Options` , `Referrer-Policy` , `Permissions-Policy` , and an explicit `Strict-Transport-Security` (Vercel also sets HSTS by default on its own, so this is redundant-but-harmless belt-and-suspenders). The CSP uses `script-src 'self' 'unsafe-inline'` and `style-src 'self' 'unsafe-inline'` deliberately — `next-themes` injects an inline bootstrap `<script>` to avoid a dark/light flash on load, and Framer Motion sets animated values via inline `style` attributes; both would be blocked by a stricter nonce-based CSP without extra wiring that wasn't judged worth it for a static site with zero user-generated content. The CSP still blocks all *remote* script/frame/object sources, which is the main realistic threat for this app. **If this CSP is ever tightened, re-run the same kind of check that validated it**: load the production build, click the theme toggle, and confirm zero console/CSP-violation errors — don't just assume it still works.
- `npm audit` **reports ~19 vulnerabilities, all in dev-only tooling** (the `shadcn` CLI and the ESLint/ `typescript-eslint` toolchain, via `deepmerge` and `minimatch` advisories). None of this code ships to the browser or runs on Vercel's serving infrastructure — it only executes locally during `npm run lint` or `npx shadcn add` . **Do not run** `npm audit fix --force` — it downgrades `eslint` to v4 (years-old, breaking, incompatible with this project's flat config) for no real risk reduction. Revisit when `shadcn` / `eslint-config-next` / `typescript-eslint` ship fixed majors upstream.
- No secrets, tokens, or keys exist anywhere in git history or the working tree (checked via full-history grep). No `dangerouslySetInnerHTML` / `eval` / `new Function` in the codebase. Every `target="_blank"` link carries `rel="noreferrer"` .
- The app has **zero environment variables and no backend** — nothing to leak via Vercel project settings.
- Not checked from this environment (needs repo-owner auth): whether GitHub's Dependabot/secret-scanning are enabled under Settings → Security on the GitHub repo. Worth a quick manual check periodically; it's a free toggle for public repos.

Deployment

- Vercel project is linked to the GitHub repo's `main` branch — **just** `git push origin main` **and it auto-deploys**, no manual `vercel` CLI step needed for routine changes.
- The project was originally linked via the Vercel CLI using a personal access token the owner generated and shared once; that token should be treated as already rotated/revoked by now and is **not** recorded anywhere in this repo. If a future AI needs to deploy via CLI again, ask the owner for a fresh token at https://vercel.com/account/tokens, or just push to `main` and let the GitHub integration handle it.
- Local commands: `npm install` , `npm run dev` (local dev), `npm run build` (production build — **always run this plus** `npm run lint` **before committing**, this project has caught several real bugs this way), `npm run start` (serve the production build locally for manual QA).

Open items / things a future session might need to follow up on

- `SmartHeal` **'s GitHub repo is unlinked.** The owner mentioned `https://github.com/Runverve-tech/SH-App` for it, but that URL 404s publicly (likely a private company repo). The SmartHeal project card currently has no "Code" link and carries an "NDA Signed" badge instead (same for Rakshak HRMS, which is genuinely NDA'd). If this repo is ever made public, add `github: "https://github.com/Runverve-tech/SH-App"` to the `smartheal` entry in `data.ts`.
- **GitHub's "Contributors" sidebar widget** on the repo's main page briefly showed a stale "claude" entry after an early commit (made with AI co-author attribution) was amended and force-pushed to remove it. Verified repeatedly that the actual git history, every commit's author/committer, and the authoritative Insights → Contributors page show only `dkwarude-cell` — this was purely a GitHub-side stats cache lag, not a real data issue, and it resolved on its own (the Insights → Contributors page corrected first, the sidebar widget caught up shortly after). **This is resolved** — flagged here only so a future session doesn't panic and force-push again if some other stale-cache artifact ever reappears; check the Insights → Contributors page first since it's more authoritative than the sidebar widget, and don't force-push without the owner's explicit sign-off (it's a destructive operation).
- No automated tests exist for this project (not requested, and a marketing/portfolio site has limited test surface). `npm run build` + `npm run lint` + manual QA (and, where feasible, a quick Playwright screenshot pass on mobile viewport) is the verification loop used throughout.

How this project was actually built (brief narrative, for context)

- Deep-research pass on what makes engineer portfolios effective for hiring (see above).
- Scaffolded with `create-next-app`, added shadcn/ui + Framer Motion, built out all sections from the owner's resume content.
- Deployed to Vercel via CLI (token-based, one-time), linked to GitHub for auto-deploy thereafter.
- Iterative refinement driven entirely by the owner's feedback: fixed a broken resume download (regenerated the PDF from scratch since only extracted text, not the original file, was ever available), wired in the real LinkedIn URL, added/reordered/relinked projects, added NDA badges, added achievement photos (with photo-cropping iterations), added "advanced/futuristic" scroll motion UI, then fixed that motion UI's mobile experience (including the Framer Motion blur bug above) and added a touch-appropriate tap-glow interaction, and iterated on the hero tagline wording several times.
- A full security review (HTTP headers, dependency audit, secret scan — see "Security" above) and a fix for email links silently failing when no mail client is configured (see gotcha 8 above), both done on request after the site was otherwise feature-complete.